IPL Cricket Data Analysis
2008 to 2024

Technical project report

Descriptive analysis, statistical inference, match-winner prediction and an interactive dashboard: the data, the packages, the algorithms, and the reasoning behind each decision.

In this report	
The data	Sources, columns, cleaning rules and edge cases
The tools	Every package and framework, and why it was chosen over alternatives
The analysis	Six descriptive analyses and six hypothesis tests, with formulas and results
The model	Elo rating, features, leakage controls, evaluation and why it barely beats a coin flip
The dashboard	Architecture, statistics computed in the browser, design and verification
The reasoning	A "why this, why that" section, a glossary and talking points

Project: ipl-cricket-analysis | Language: Python 3 and JavaScript | Report date: 23 October 2026

Contents

1. Project at a glance **4**

Headline results . . . 4

2. Project structure **5**

How the pieces fit together . . . 5

How to run everything . . . 5

3. The data **6**

3.1 Source . . . 6

3.2 The two files . . . 6

3.3 Cleaning rules and why . . . 6

3.4 Definitions used throughout . . . 7

4. Technology stack **8**

4.1 Packages considered and not used . . . 8

5. Descriptive analysis (notebook 1) **9**

5.1 Has scoring changed? . . . 9

5.2 Which phase of the innings changed most? . . . 9

5.3 Toss and chasing . . . 10

5.4 Team records . . . 10

5.5 Top performers . . . 11

5.6 Is Chepauk special for CSK? . . . 11

6. Statistical inference (notebook 2) **13**

6.1 Concepts used . . . 13

6.2 Methods and formulas . . . 13

6.3 The six tests . . . 14

6.4 Summary after correction . . . 16

7. Predicting the winner (notebook 2) **17**

7.1 Problem framing . . . 17

7.2 Why a time-based split . . . 17

7.3 Team strength with an Elo rating . . . 17

7.4 Features . . . 18

7.5 Leakage controls . . . 18

7.6 Models and baselines . . . 18

7.7 Metrics . . . 19

7.8 Results on the 2023 and 2024 test set . . . 19

7.9 Walk-forward evaluation . . . 19

7.10 Are the probabilities honest? Calibration . . . 20

7.11 What drives the predictions? . . . 21

7.12 Why the model barely beats a coin flip . . . 22
8. The interactive dashboard 23
8.1 Purpose . . . 23
8.2 Architecture . . . 23
8.3 What is on the page . . . 23
8.4 Design decisions . . . 24
8.5 Rebuilding . . . 24
9. Verification performed 25
What was not verified . . . 25
10. Reproducibility 25
11. Limitations and threats to validity 26
Future work . . . 26
12. Why this, why that 27
13. Glossary 29
14. Talking points for a resume or interview 30
Resume bullets . . . 30
Three points to be ready to defend . . . 30
Appendix A. Code for the key statistical helpers 31
Wilson interval . . . 31
Percentile bootstrap for a difference . . . 31
Holm-Bonferroni correction . . . 31
Permutation test for the Chepauk gap . . . 31
Evaluation of one walk-forward fold . . . 31

1. Project at a glance

This project uses every Indian Premier League (IPL) match from 2008 to 2024, 1,095 matches and about 260,000 ball-by-ball records, to test things people often say about the league: that scores keep getting bigger, that the toss decides matches, and that Chepauk is a fortress for Chennai Super Kings. It then goes one step further than description by asking which of those claims are statistically real and whether match winners can be predicted before the first ball.

Item	Detail
Questions	How have scores changed? Does the toss matter? Is chasing better? Which teams and players lead? Is Chepauk special? Which findings survive statistical testing? Can the winner be predicted?
Data	Cricsheet ball-by-ball data via the Kaggle dataset "IPL Complete Dataset (2008-2024)": 1,095 matches, about 260,000 deliveries, cleaned and trimmed.
Notebook 1: description	Notebook ipl_analysis.ipynb. Scoring trend, phase run rates, toss and chasing, franchise records, top players, Chepauk. Charts 01 to 06.
Notebook 2: inference and prediction	Notebook ipl_inference_and_prediction.ipynb. Six hypothesis tests with confidence intervals and a Holm correction, an Elo-based win predictor, walk-forward evaluation. Charts 07 to 11.
Dashboard	docs/index.html. A self-contained interactive page with season filters, team and ground explorers, live confidence intervals and the notebook 2 results.
Tools	Python, pandas, NumPy, SciPy, scikit-learn, Matplotlib, seaborn, Jupyter; plain HTML, CSS and JavaScript with SVG for the dashboard.

Headline results

- **Scoring jumped.** The average first-innings score rose from 161 in 2008 to 190 in 2024. Most of the rise came in 2023 and 2024: about 163 runs for 2008 to 2022 against about 186 for 2023 and 2024, a difference of +23.1 runs (95% interval +17.0 to +29.1, Cohen's d = 0.75).
- **The toss does not matter, as far as this data can tell.** Toss winners won 50.8% of matches (95% interval 47.9% to 53.8%).
- **Chasing has a small real edge.** The chasing team won 54.1% of decided matches (95% interval 51.2% to 57.1%).
- **Home advantage exists league-wide** (56.5%, interval 52.5% to 60.4%). CSK at Chepauk (70.4%) and batting first at Chepauk (57.6%) are real but fragile findings: they pass the multiple-testing correction narrowly and their intervals are wide.
- **Match winners are barely predictable from pre-match information.** On 144 test matches in 2023 and 2024 no model could be told apart from a coin flip. Over nine walk-forward seasons the best fitted model reached 54.7% accuracy.

Why the negative prediction result is a good result
A prediction model that beats a coin flip by a small margin, reported with intervals, baselines and a leak-free evaluation, is stronger evidence of method than an inflated accuracy figure. T20 cricket is high-variance, squads change every year, and teams are closely matched; this report shows how that shows up in the numbers.

2. Project structure

Path	What it is	Why it exists
ipl_analysis.ipynb	Notebook 1, outputs saved	Descriptive analysis; readable directly on GitHub
ipl_inference_and_prediction.ipynb	Notebook 2, outputs saved	Hypothesis tests and win prediction; ends by writing results/results.json
data/matches_2008-2024.csv	One row per match (20 columns)	Match-level facts: teams, toss, venue, winner
data/deliveries_2008-2024.csv.gz	One row per ball (15 columns, compressed)	Ball-by-ball facts: runs, wickets, bowlers, batters
charts/	11 PNG files	Figures saved by the notebooks and used in the README
results/results.json	Numbers saved by notebook 2	Single source of truth for the dashboard's test and model results
dashboard/template.html	Dashboard source	HTML, CSS and JavaScript with a data placeholder
scripts/build_dashboard.py	Build script	Combines data, results and template into docs/index.html
docs/index.html	Built dashboard	Folder name chosen so GitHub Pages can serve it
requirements.txt	Package list	One-command environment setup
README.md, LICENSE, .gitignore	Project metadata	Explanation, MIT licence, and files git should skip

How the pieces fit together

```
raw CSV files (data/)
   |  clean: strip spaces, merge renamed franchises, drop super overs
   v
ipl_analysis.ipynb ----------------------> charts/01 ... 06
ipl_inference_and_prediction.ipynb --------> charts/07 ... 11
   |                                          and results/results.json
   v
scripts/build_dashboard.py  (data + results.json + dashboard/template.html)
   v
docs/index.html  ---- GitHub Pages ----> public interactive dashboard
```

How to run everything

```
git clone https://github.com/YOUR-USERNAME/ipl-cricket-analysis.git
cd ipl-cricket-analysis
pip install -r requirements.txt
jupyter notebook ipl_analysis.ipynb                      # notebook 1, writes charts 01-06
jupyter notebook ipl_inference_and_prediction.ipynb      # notebook 2, ~1 minute, writes charts 07-11
python scripts/build_dashboard.py                        # rebuilds docs/index.html
```

The second notebook takes about a minute and uses a fixed random seed (42), so its results reproduce exactly when the cells are run from top to bottom in order. The order matters because one random-number generator is shared across cells (see section 10).

3. The data

3.1 Source

Ball-by-ball data comes from **Cricsheet**, packaged in the Kaggle dataset "IPL Complete Dataset (2008-2024)". The files were cleaned, trimmed to the columns needed and compressed so they fit on GitHub. Cricsheet's terms should be checked before the data is reused.

3.2 The two files

File	Rows	Key columns
matches_2008-2024.csv	1,095	id, season, city, date, match_type, player_of_match, venue, team1, team2, toss_winner, toss_decision, winner, result, result_margin, target_runs, target_overs, super_over, method, umpire1, umpire2
deliveries_2008-2024.csv.gz	about 260,000	match_id, inning, batting_team, bowling_team, over, ball, batter, bowler, batsman_runs, extra_runs, total_runs, extras_type, is_wicket, player_dismissed, dismissal_kind

Match outcomes in the data

Result type	Matches	Match type	Matches
Won by wickets	578	League	1,029
Won by runs	498	Final	17
Tie (decided by super over)	14	Qualifier 1 / Qualifier 2	14 / 14
No result	5	Eliminator / Semi Final / Elimination Final	11 / 6 / 3
Rain-adjusted target (D/L)	21	3rd place play-off	1

Matches per season: 2008: 58, 2009: 57, 2010: 60, 2011: 73, 2012: 74, 2013: 76, 2014: 60, 2015: 59, 2016: 60, 2017: 59, 2018: 60, 2019: 60, 2020: 60, 2021: 60, 2022: 74, 2023: 74, 2024: 71. Season length varies with the number of teams in the league.

3.3 Cleaning rules and why

Rule	Why
Strip leading and trailing spaces from every column name and every text value	The original files had extra spaces in column names and values; unmatched strings silently break joins and group-bys.
Rename id to match_id in the matches table	Makes the join key identical in both tables.
Merge renamed franchises: Delhi Daredevils into Delhi Capitals, Kings XI Punjab into Punjab Kings, Royal Challengers Bangalore into Royal Challengers Bengaluru, Rising Pune Supergiants into Rising Pune Supergiant	These are the same franchise under different names; leaving them split would cut each team's sample in half and bias win rates.
Keep Deccan Chargers and Sunrisers Hyderabad separate	They are different franchises that happened to play in the same city at different times. Pune Warriors, Kochi Tuskers Kerala, Gujarat Lions, Gujarat Titans and Lucknow Super Giants are likewise separate.
Keep only innings 1 and 2 in the ball-by-ball data	Innings 3 and above are super overs, which follow different rules and would distort run rates.
Win-rate analyses use only matches with a recorded winner (1,090 of 1,095)	Five matches had no result; they carry no information about who wins.
Exclude rain-adjusted (D/L) first innings from the scoring hypothesis test only	Shortened innings are not comparable. The descriptive charts keep all matches so they match the first notebook.
Merge spelling variants of venues (for example three spellings of Chepauk)	The same ground appears under up to three names; without merging, venue counts would be split and Chepauk would be missed.

3.4 Definitions used throughout

Term	Definition
First-innings score	Sum of total_runs (bat runs plus extras) over all balls of innings 1 in a match.
Legal ball	A delivery that is not a wide or a no-ball; used to count overs for run rates.
Phase of play	Powerplay: overs 1 to 6 (over index 0 to 5). Middle: overs 7 to 15 (6 to 14). Death: overs 16 to 20 (15 to 19).
Run rate	Total runs in a phase divided by legal balls in that phase divided by 6, i.e. runs per over.
Decided match	A match with a recorded winner (1,090 matches).
Chasing team won	Toss winner chose to field and won, or toss winner chose to bat and lost.
Batting-first team	The toss winner if they chose to bat; otherwise the other team.
Wicket credited to the bowler	A dismissal that is not run out, retired hurt, retired out or obstructing the field.
Title	A win in a match of type Final.

4. Technology stack

The stack was chosen to be standard, free, installable with one command and easy for a reviewer to run. Versions below are the ones the project was built and tested with (Python 3.13); requirements.txt asks only for recent minimum versions.

Package	Tested version	Role in the project	Why this one
Python	3.13	Language for all analysis and the build script	Standard for data science; readable for reviewers
pandas	3.0.5 (needs 2.2+)	Loading, cleaning, joining and grouping about 260,000 rows	Industry standard for tabular data; groupby and merge express the analysis directly
NumPy	2.5.3 (needs 1.26+)	Random numbers, array maths, resampling	Fast vector operations; seeded Generator for reproducibility
SciPy	1.18.1 (needs 1.11+)	binomtest, fisher_exact, mannwhitneyu, normal quantiles	Well-tested exact statistical tests without extra dependencies
scikit-learn	1.9.1 (needs 1.4+)	LogisticRegression, HistGradientBoostingClassifier, StandardScaler, pipelines, metrics, permutation importance	One consistent API for models, scaling and evaluation
Matplotlib	3.11.2 (needs 3.8+)	Static charts saved as PNG	Full control over annotated figures
seaborn	0.13.2 (needs 0.13+)	Chart theme (whitegrid) and style	Clean defaults on top of Matplotlib
Jupyter	latest	Notebooks that mix code, charts and explanation	GitHub renders saved notebooks in the browser
HTML, CSS, JavaScript (no framework)	n/a	The interactive dashboard, with hand-drawn SVG charts	No server, no build tooling, no runtime dependency: it opens from a file and runs on GitHub Pages
Google Fonts (Barlow)	n/a	Typeface for the dashboard	Condensed scoreboard-style headings; falls back to system fonts if offline

4.1 Packages considered and not used

Alternative	Why not
statsmodels	Every test needed (binomial, Fisher, Mann-Whitney) is in SciPy, and the Holm correction is ten lines of code, written by hand and shown in section 6. One fewer dependency.
Deep learning (PyTorch, TensorFlow)	There are about 1,000 training matches with weak signal. A neural network would overfit and add no insight. The gradient-boosting result in section 7 shows even a modest flexible model overfits.
XGBoost or LightGBM	scikit-learn's HistGradientBoostingClassifier is the same family of model, built in, and sufficient for this data size.
SHAP	Permutation importance is model-agnostic, built into scikit-learn and easy to explain; it gives the same kind of ranking for this small feature set.
Streamlit, Dash or Plotly	These need a running Python server or a heavy JavaScript library. A static page works on free GitHub Pages and never breaks when a dependency changes.
Tableau or Power BI	Proprietary and not versionable in a code repository.

5. Descriptive analysis (notebook 1)

The first notebook answers six questions from the data. Each subsection gives the method, the reason for it and the result.

5.1 Has scoring changed?

Method. For every match, sum total_runs over innings 1 to get the first-innings score; count sixes (batsman_runs equal to 6) over both innings. Average by season. **Why.** The mean is the natural summary for a total, and a per-season line shows both the trend and the size of year-to-year noise.

Result. The average first-innings score rose from 161 (2008) to 190 (2024). The average of 2008 to 2022 was about 163 against about 186 for 2023 and 2024. Sixes per match rose from 10.7 to 17.8.

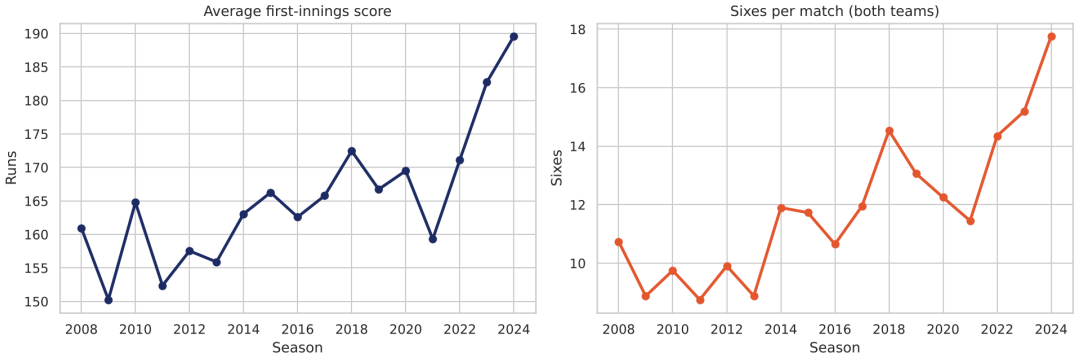


Figure 1. Average first-innings score and sixes per match, by season (chart 01).

5.2 Which phase of the innings changed most?

Method. Split every ball by over into powerplay, middle and death. For each season and phase, divide runs by legal balls divided by 6. **Why.** Dividing by legal balls rather than by balls bowled stops wides and no-balls from understating the rate; splitting by phase shows where the extra scoring comes from.

Result. Comparing the 2008 to 2022 average with 2024, the powerplay run rate rose from 7.69 to 9.46 runs per over (+1.77), the middle overs from 7.67 to 8.83 (+1.16) and the death overs from 9.83 to 11.21 (+1.38). The powerplay changed the most.

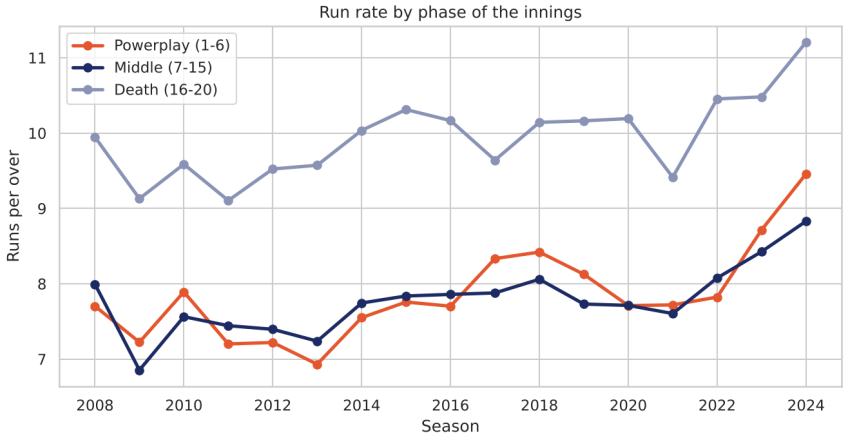


Figure 2. Run rate by phase of the innings (chart 02).

5.3 Toss and chasing

Method. Among 1,090 decided matches, compute how often the toss winner won, how often the chasing team won, and how often toss winners chose to field. **Why.** Two separate questions are hidden in "does the toss matter": whether winning the toss helps, and whether the side batting second has an advantage.

Result. Toss winners won 50.8% of matches, almost a coin flip. The chasing team won 54.1%, and chasing teams won more than half of matches in 12 of 17 seasons. Captains noticed: toss winners chose to field 43% of the time in 2008 to 2010 and 75% in 2022 to 2024. Section 6 puts confidence intervals on the first two figures.

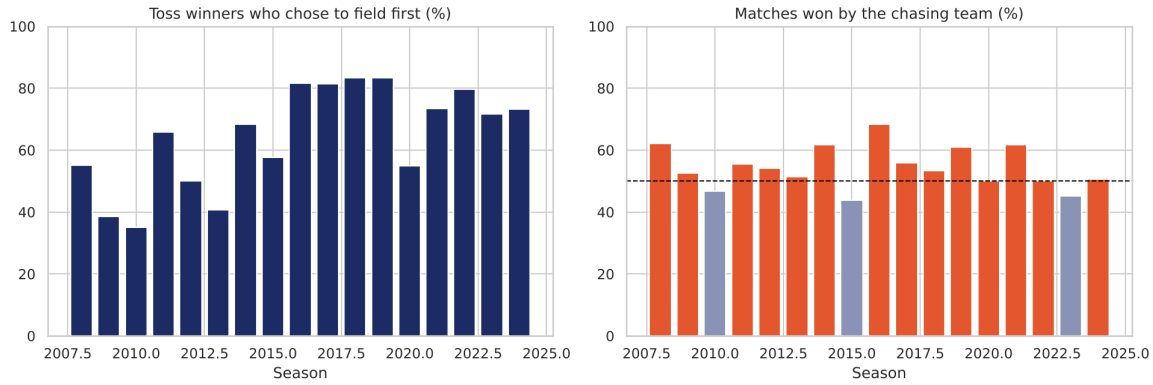


Figure 3. Toss winners choosing to field (left) and matches won by the chasing team (right), by season (chart 03).

5.4 Team records

Method. For franchises with at least 50 matches with a result, divide wins by matches; count titles as wins in matches of type Final. **Why.** The 50-match floor keeps tiny samples (for example Kochi Tuskers Kerala, one season) out of the ranking.

Franchise	Matches with a result	Win rate	Titles
Chennai Super Kings	237	58.2%	5
Mumbai Indians	261	55.2%	5
Kolkata Knight Riders	251	52.2%	3
Rajasthan Royals	219	51.1%	1
Royal Challengers Bengaluru	252	48.8%	0
Sunrisers Hyderabad	182	48.4%	1
Delhi Capitals	250	46.0%	0
Punjab Kings	246	45.5%	0
Deccan Chargers	75	38.7%	1

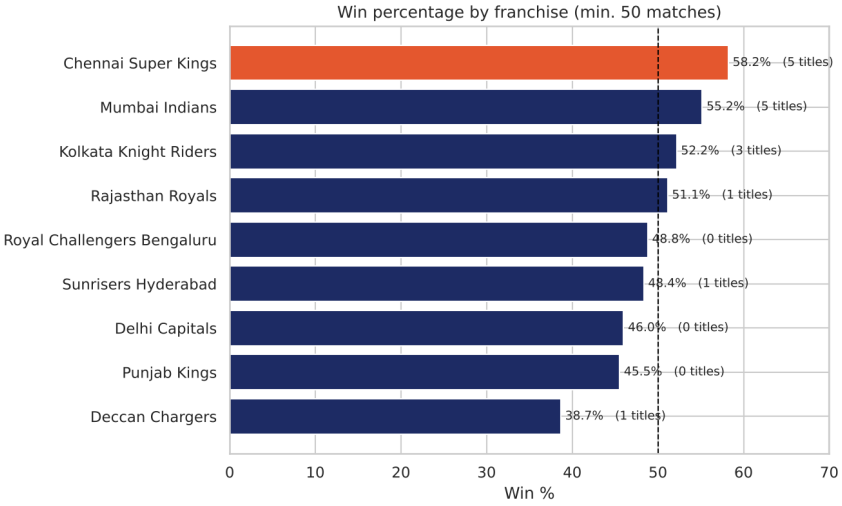


Figure 4. Win percentage by franchise with title counts (chart 04).

5.5 Top performers

Method. Runs are the sum of batsman_runs (extras are excluded because they are not credited to the batter). Wickets count dismissals credited to the bowler. The code also computes strike rate as runs divided by balls faced excluding wides. **Result.** Virat Kohli leads run scorers with 8,004 runs, ahead of Shikhar Dhawan (6,769) and Rohit Sharma (6,628). Yuzvendra Chahal leads wicket takers with 205, ahead of Piyush Chawla (192) and Dwayne Bravo (183).

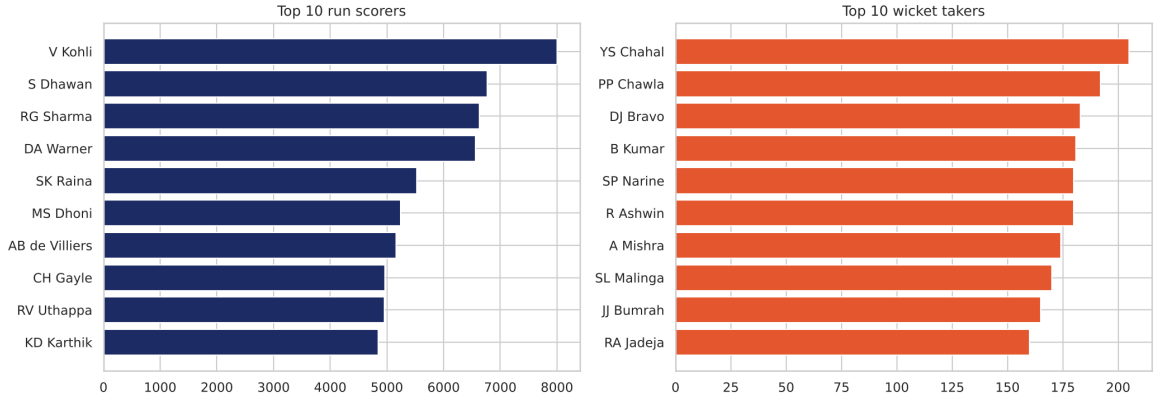


Figure 5. Top ten run scorers and wicket takers, 2008 to 2024 (chart 05).

5.6 Is Chepauk special for CSK?

Method. Compare CSK's win rate at Chepauk with its win rate elsewhere, and compare the batting-first win rate at Chepauk with all other grounds. **Result.** CSK won 70.4% at Chepauk (50 of 71) against 53.0% elsewhere (88 of 166). Chepauk has 85 matches with a result. The team batting first won 57.6% at Chepauk but only 45.9% across all venues, so the usual "chasing is better" pattern reverses there. The average first-innings score at Chepauk (164) is close to the all-venue average (166), so this is not simply a low-scoring ground.

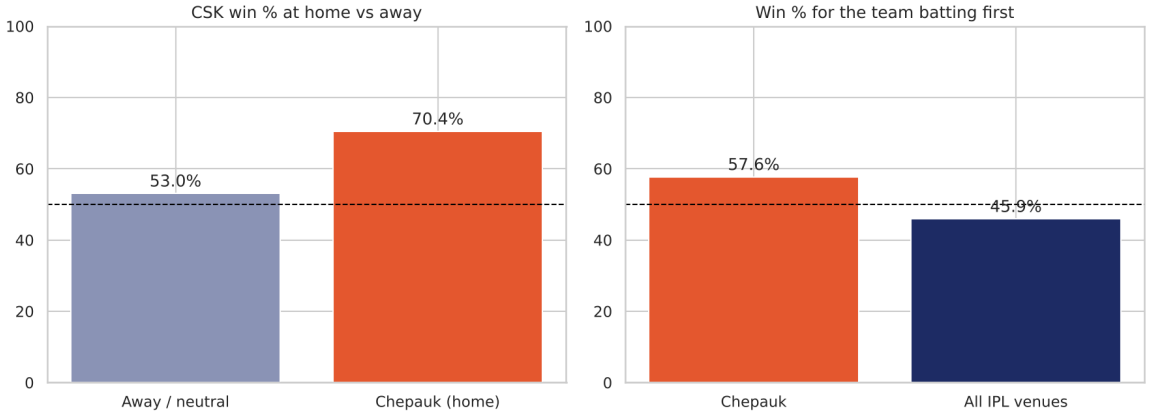


Figure 6. CSK win rate at home and away (left); batting-first win rate at Chepauk and across all venues (right) (chart 06).

Limit of descriptive analysis

These are percentages with no measure of uncertainty. A 70.4% record from 71 matches could shift with a few games. That gap is what section 6 addresses.

6. Statistical inference (notebook 2)

6.1 Concepts used

Idea	Plain meaning
Null hypothesis (H0)	The "nothing is going on" claim, for example "the toss winner wins 50% of the time".
p-value	How surprising the data would be if H0 were true. Small values are evidence against H0; they are not the probability that H0 is true.
95% confidence interval (CI)	A range of plausible values for the true effect. If a CI for a win rate crosses 50%, the data is consistent with a coin flip.
Effect size	How big the difference is, regardless of sample size. A tiny effect can be "significant" in a huge sample; a large effect can be "insignificant" in a small one.
Power and minimum detectable effect	The smallest true effect a sample can usually detect. Here it is about 4.2 points for the toss test (80% power).
Multiple testing	Running several tests raises the chance that at least one false positive appears; p-values must be adjusted together.

6.2 Methods and formulas

Method	Used for	How it works	Why chosen
Exact binomial test	Toss, chasing, league-wide home advantage	Computes the exact probability of a result at least as extreme as observed if the true rate is 0.5.	Exact for a yes/no outcome; no large-sample approximation needed.
Wilson score interval	Every CI on a proportion	Centre = $(p + z^2/2n) / (1 + z^2/n)$; half-width = $z * \text{sqrt}(p(1-p)/n + z^2/4n^2) / (1 + z^2/n)$, $z = 1.96$.	Stays inside 0 to 1 and behaves well for small samples, unlike the simple p +/- 1.96 se formula.
Fisher's exact test	CSK home vs away; chasing early vs late era	Exact p-value for a 2 by 2 table of counts.	Cell counts are small, so the chi-squared approximation is unreliable.
Percentile bootstrap (10,000 resamples)	CI for a difference in win rates or in mean scores	Resample each group with replacement, recompute the difference, take the 2.5th and 97.5th percentiles.	Makes no distributional assumption and works for any statistic.
Permutation test (20,000 shuffles)	Batting-first win rate at Chepauk vs elsewhere	Shuffle the "Chepauk" label, recompute the gap each time, count how often a gap as large as the observed one appears.	Directly answers "could labels at random produce this gap?" with no formula.
Mann-Whitney U	Scores 2023-24 vs 2008-22	Rank-based test of whether one group tends to be larger.	Scores are not exactly normal, and the test is robust to outliers.
Cohen's d	Size of the scoring jump	Difference in means divided by the pooled standard deviation.	A unit-free effect size; 0.75 is conventionally a medium-to-large effect.
Holm-Bonferroni correction	All six tests together	Sort p-values ascending; adjusted $p_{(i)}$ = max over j up to i of $(m - j + 1) * p_{(j)}$, capped at 1.	Controls the chance of any false positive, and is never less powerful than plain Bonferroni.

6.3 The six tests

Test 1. Does winning the toss help?

H0: the toss winner wins 50% of matches. The toss winner won 554 of 1,090 (50.8%), 95% CI 47.9% to 53.8%, p = 0.61. There is no evidence of an effect. With 1,090 matches the test could usually detect a true advantage of about 4.2 points or more (minimum detectable effect = $(z_{0.975} + z_{0.80}) * 0.5 / \sqrt{n}$), so a smaller advantage cannot be ruled out. Absence of evidence is not evidence of absence.

Test 2. Is chasing better than batting first?

H0: the chasing team wins 50%. The chasing team won 590 of 1,090 (54.1%), 95% CI 51.2% to 57.1%, p = 0.007 (Holm-adjusted 0.028). A small real edge. A check on whether it changed over time found 53.5% in 2008 to 2015 and 54.7% in 2016 to 2024 (Fisher exact p = 0.715), so the edge has been stable.

Test 3. Is CSK better at Chepauk? (and Test 3b: is any team better at home?)

H0: CSK's win probability is the same at Chepauk and elsewhere. CSK won 50 of 71 at Chepauk (70.4%) against 88 of 166 elsewhere (53.0%): a difference of +17.4 points, bootstrap 95% CI +4.3 to +30.3, Fisher p = 0.015 (Holm 0.044).

Because CSK is also a strong team overall, part of that gap is team quality rather than the ground. The league-wide question has far more data: across all teams, home sides beat visitors in 342 of 605 matches (56.5%), 95% CI 52.5% to 60.4%, p = 0.0015 (Holm 0.0075). CSK's Chepauk record is also above that league-wide home rate (50 of 71 against 56.5%, p = 0.02, not corrected), so Chepauk looks better than ordinary home advantage. This rests on one franchise and 71 matches, and the data cannot separate ground from team quality.

Team	Home games vs visitors	Home win %
Chennai Super Kings	71	70.4
Rajasthan Royals	57	64.9
Sunrisers Hyderabad	57	63.2
Mumbai Indians	85	61.2
Kolkata Knight Riders	88	59.1
Punjab Kings	61	50.8
Royal Challengers Bengaluru	87	50.6
Delhi Capitals	81	45.7
Deccan Chargers	18	16.7

The home-ground mapping used: CSK at Chidambaram, MI at Wankhede, KKR at Eden Gardens, RCB at Chinnaswamy, Delhi at Feroz Shah Kotla or Arun Jaitley, Punjab at Mohali (PCA), Rajasthan at Sawai Mansingh, Sunrisers and Deccan Chargers at Rajiv Gandhi (Uppal). Matches where both teams count that ground as home are excluded.

Test 4. Is batting first different at Chepauk?

H0: the batting-first win rate at Chepauk equals that at all other grounds. Chepauk 57.6% (n = 85) against 44.9% elsewhere (n = 1,005): +12.8 points, bootstrap 95% CI +1.7 to +23.8, permutation p = 0.023 (Holm 0.047). A narrow pass with a wide interval.

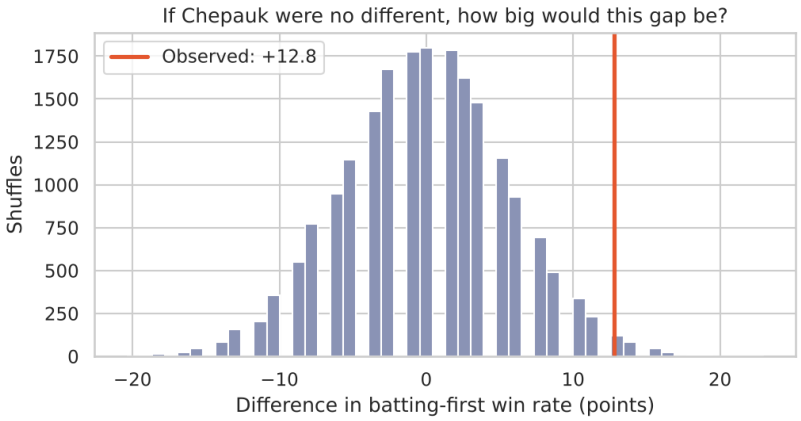


Figure 7. If the Chepauk label were meaningless, shuffling it 20,000 times rarely produces a gap as large as the observed +12.8 points (chart 07).

Test 5. Did scoring really jump in 2023 and 2024?

H0: first-innings scores in 2023 and 2024 come from the same distribution as 2008 to 2022. Mean 162.8 (n = 931) against 185.9 (n = 143): +23.1 runs, bootstrap 95% CI +17.0 to +29.1, Mann-Whitney p = 1.7e-13 (Holm 1e-12), Cohen's d = 0.75. The jump is large and certain. The **Impact Player rule**, introduced in 2023, is one plausible cause because it lengthens batting line-ups, but pitches, player mix and tactics also changed and this data cannot separate them.

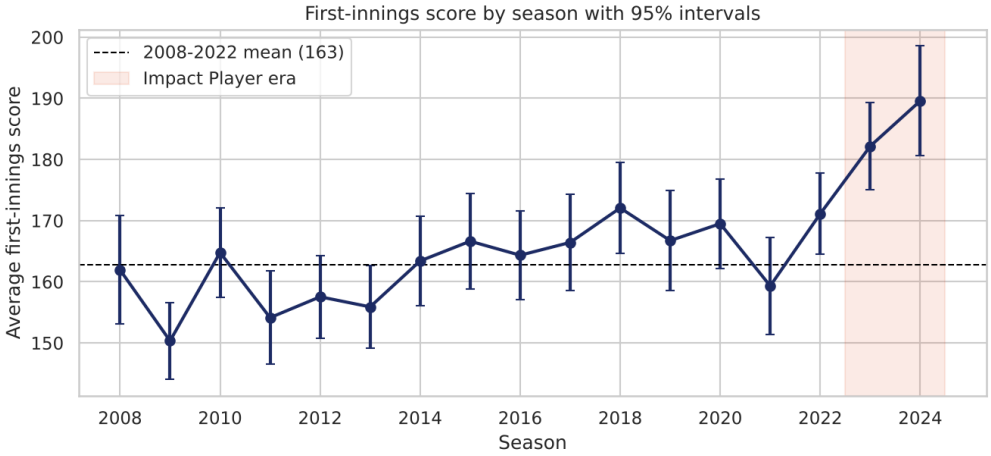


Figure 8. First-innings score by season with 95% intervals; the 2023 and 2024 intervals sit above the 2008 to 2022 mean (chart 08).

6.4 Summary after correction

Claim	Test	Effect	95% CI	Holm p	Verdict
Toss winner wins the match (vs 50%)	Binomial	50.8%	47.9 to 53.8	0.61	No clear evidence
Chasing team wins (vs 50%)	Binomial	54.1%	51.2 to 57.1	0.028	Real effect
CSK win rate: Chepauk minus elsewhere	Fisher exact	+17.4 pts	+4.3 to +30.3	0.044	Real effect
Any team wins at own home ground (vs 50%)	Binomial	56.5%	52.5 to 60.4	0.0075	Real effect
Batting-first win rate: Chepauk minus elsewhere	Permutation	+12.8 pts	+1.7 to +23.8	0.047	Real effect
First-innings score: 2023-24 minus 2008-22	Mann-Whitney	+23.1 runs	+17.0 to +29.1	1e-12	Real effect

Five of six claims pass the 5% threshold after correction; the toss does not. Two of the five (CSK at Chepauk and batting first at Chepauk) pass only narrowly and have wide intervals, so they should be described as real but fragile.

7. Predicting the winner (notebook 2)

7.1 Problem framing

- **Unit of analysis:** one decided match (1,090). **Target:** 1 if "team A" won.
- **Orientation is randomised.** Which team is team A is chosen at random with a fixed seed. Without this, a model could learn from the order teams happen to be listed in the file (for example if team1 tended to bat first), which would be a form of leakage.
- **Information allowed:** only what is known before the first ball, including the toss. Nothing from the match itself.
- **Split:** train on 2008 to 2022 (946 matches), test on 2023 and 2024 (144 matches). Share of matches won by team A: 0.499 in training and 0.486 in testing, close to 0.5 as intended.

7.2 Why a time-based split

Predicting the future means training on the past. A random split would place matches from the same season in both training and test sets, so the model could pick up season-specific patterns and overstate accuracy. Splitting by time mimics real use: fit on what is known, then predict what has not happened.

7.3 Team strength with an Elo rating

Elo is a rating system from chess. Every team starts at 1500. Before a match, the expected score of team 1 is $E_1 = 1 / (1 + 10^{(R_2 - R_1)/400})$. After the match, each rating moves by K * (actual - expected), where actual is 1 for a win and 0 for a loss. Ratings are updated only after a match, so a match's pre-game rating reflects earlier matches only. At each new season, ratings are pulled toward 1500: R becomes 1500 + c * (R - 1500). This reflects that squads change at every auction.

```
def run_elo(df, k=20, season_carry=0.75, base=1500.0):
    ratings, last_season = {}, None
    for _, m in df.sort_values(["date", "match_id"]).iterrows():
        if last_season is not None and m.season != last_season:       # new season: shrink to the mean
            ratings = {t: base + season_carry * (r - base) for t, r in ratings.items()}
        last_season = m.season
        r1, r2 = ratings.get(m.team1, base), ratings.get(m.team2, base)
        # (pre-match ratings r1, r2 are recorded here, BEFORE the update below)
        if pd.notna(m.winner):
            exp1 = 1 / (1 + 10 ** ((r2 - r1) / 400))
            s1 = 1.0 if m.winner == m.team1 else 0.0
            ratings[m.team1] = r1 + k * (s1 - exp1)
            ratings[m.team2] = r2 + k * ((1 - s1) - (1 - exp1))
```

Choosing K and the season carry-over. Twelve combinations (K in 5, 10, 20, 30; carry-over in 0.5, 0.75, 1.0) were scored by log-loss on the 2016 to 2022 matches, never on the 2023 and 2024 test years. The best was K = 5 with carry-over 0.75 (log-loss 0.6934). A coin flip scores 0.6931. The best Elo is therefore no better than chance, and a small K, meaning very slow rating changes, won. That is itself a finding: team strength differences are small relative to the randomness of a single match.

K	Season carry-over	Validation log-loss (2016 to 2022)
5	0.75	0.6934
5	0.50	0.6937
10	0.50	0.6952
10	0.75	0.6955
5	1.00	0.6969

7.4 Features

Feature	Definition	Why included
elo_diff	Elo of team A minus Elo of team B, from run_elo with K = 5 and carry 0.75	Overall team strength from all earlier results
form_diff	Smoothed win rate over each team's last 5 matches, (wins + 1.5) / (matches + 3), team A minus team B	Recent form; smoothing pulls tiny samples toward 0.5
career_diff	Smoothed career win rate, (wins + 5) / (matches + 10), team A minus team B	Long-run strength; heavier smoothing than form
home_diff	1 if only A plays at its home ground, -1 if only B does, 0 otherwise	Tests the home advantage found in section 6
toss_a	1 if team A won the toss	Known at the toss; lets the model use any toss effect
bat_first_edge	(2 * A_bats_first - 1) * (venue batting-first win rate - 0.5), the venue rate smoothed toward 0.46 with weight 10	Some grounds favour the side batting first; this signs that edge by who bats first

7.5 Leakage controls

- Elo, form, career record and venue statistics are all computed from matches strictly before the one being predicted, and updated only after it.
- Team order is randomised so list order carries no information.
- Hyperparameters of the Elo rating were chosen on 2016 to 2022 only; the 2023 and 2024 hold-out was not used for any choice.
- The toss is used, which is legitimate because it is known before the first ball, though not before the toss.

7.6 Models and baselines

Model	Configuration	Why
Coin flip	Probability 0.5 for every match	The floor any model must beat
Better record so far	Pick the team with the higher smoothed career win rate; probability 0.55 or 0.45	A simple rule a fan would use; modest probabilities so log-loss is fair
Back the home team	Probability 0.5 + 0.05 * home_diff	Tests whether home advantage alone is enough
Elo only	Win probability straight from the Elo formula, no fitting	Tests the rating on its own
Logistic regression	StandardScaler then LogisticRegression(C = 0.5, max_iter = 1000)	Simple, regularised, with readable coefficients
Gradient boosting	HistGradientBoostingClassifier(max_depth = 2, learning_rate = 0.05, max_iter = 150, l2_regularization = 5, min_samples_leaf = 30, random_state = 42)	A flexible model, deliberately kept shallow and regularised because there are only about 1,000 training matches

7.7 Metrics

- **Accuracy:** share of matches where the predicted winner was right. Easy to read, but it ignores confidence and, with 144 matches, has a standard error near 4 points.
- **Log-loss:** the average of -[y log p + (1 - y) log(1 - p)]. It rewards honest probabilities and punishes confident mistakes. A coin flip scores 0.693; lower is better.
- **Brier score:** the average squared gap between probability and outcome. A coin flip scores 0.250; lower is better.
- **Bootstrap interval:** 5,000 resamples of the test matches give a 95% interval for each accuracy and for each model's log-loss gain over a coin flip.

7.8 Results on the 2023 and 2024 test set

Model	Accuracy	Log-loss	Brier
Coin flip	51.4%	0.693	0.250
Better record so far	56.9%	0.684	0.246
Back the home team	50.7%	0.694	0.251
Elo only	52.1%	0.694	0.250
Logistic regression	52.1%	0.704	0.255
Gradient boosting	50.7%	0.709	0.257

Model	Accuracy	Accuracy 95% CI	Log-loss gain vs coin	Gain 95% CI
Better record so far	56.9%	49% to 65%	+0.009	-0.008 to +0.024
Elo only	52.1%	44% to 60%	-0.001	-0.008 to +0.006
Logistic regression	52.1%	44% to 60%	-0.011	-0.035 to +0.012
Gradient boosting	50.7%	42% to 59%	-0.015	-0.048 to +0.015

Reading it. Every accuracy interval includes 50% and every log-loss gain interval includes zero, so no model can be told apart from a coin flip on this test set. The simple "better record so far" rule scored highest (56.9%), but its interval is 49% to 65% and it falls to 53.5% in the longer walk-forward test below, so that lead looks like luck. Gradient boosting was the worst model.

7.9 Walk-forward evaluation

One test set is a single draw. To check stability, the same exercise was repeated for every season from 2016 to 2024: train on all earlier seasons, test on that season, refit the model each time. This gives 576 test matches across nine seasons.

Model	Matches	Accuracy	Mean log-loss
Coin flip	576	48.4%	0.693
Better record so far	576	53.5%	0.691
Elo only	576	52.4%	0.693
Logistic regression	576	54.7%	0.692
Gradient boosting	576	52.4%	0.708

Model	2016	2017	2018	2019	2020	2021	2022	2023	2024
Coin flip	0.693	0.693	0.693	0.693	0.693	0.693	0.693	0.693	0.693
Better record so far	0.686	0.688	0.671	0.700	0.705	0.695	0.706	0.675	0.694
Elo only	0.694	0.689	0.690	0.694	0.694	0.691	0.701	0.690	0.699
Logistic regression	0.692	0.683	0.666	0.673	0.701	0.683	0.714	0.723	0.691
Gradient boosting	0.757	0.688	0.668	0.699	0.693	0.702	0.731	0.716	0.714

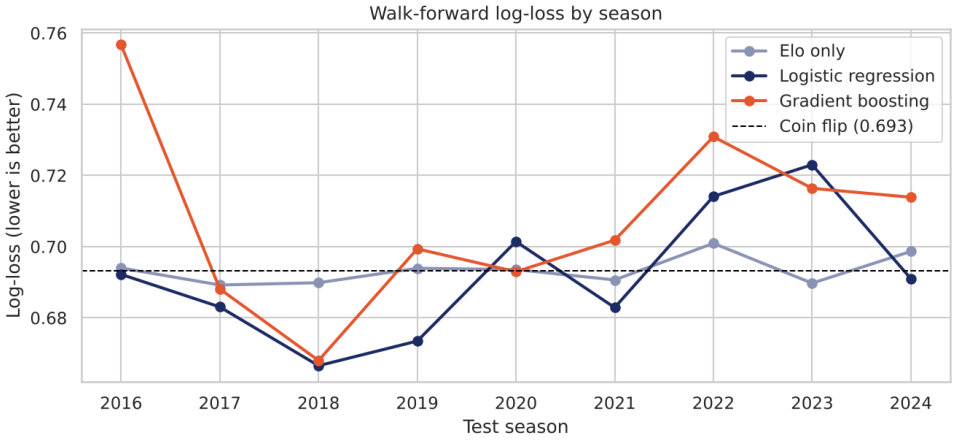


Figure 9. Log-loss by test season; the dashed line is a coin flip (chart 09).

Logistic regression sits very slightly under the coin flip on average (0.692 against 0.693) and reaches 54.7% accuracy. It beats a coin in some seasons (2017 to 2019 and 2021) and loses in others (2022 and 2023), which is what noise around zero edge looks like. Gradient boosting is clearly worse, which is typical when a flexible model meets few, noisy examples.

7.10 Are the probabilities honest? Calibration

Predicted probability bin	Average predicted	Actual win rate	Matches
0 to 0.35	0.321	0.300	10
0.35 to 0.45	0.412	0.512	121
0.45 to 0.55	0.499	0.495	297
0.55 to 0.65	0.585	0.585	142
0.65 to 1	0.668	0.333	6

Most predictions fall between 0.45 and 0.65, where predicted and actual rates agree closely. The top bin (6 matches) and bottom bin (10 matches) are too small to read anything from.

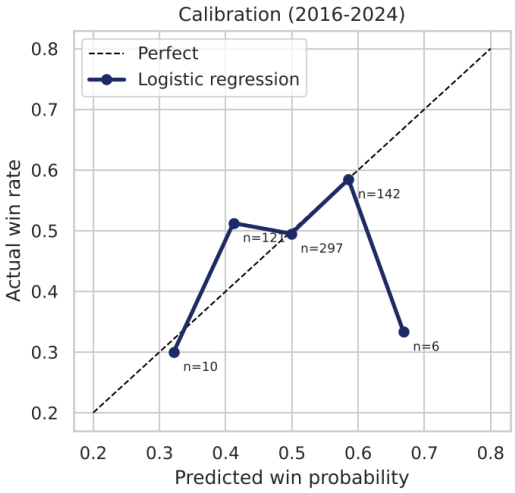


Figure 10. Calibration of the logistic regression across the walk-forward seasons (chart 10).

7.11 What drives the predictions?

Two views were used. Logistic coefficients on standardised features show direction and size. Permutation importance shuffles one feature at a time on the test set and measures how much log-loss worsens, which works for any model. Both put **home advantage** first, then **Elo difference**. Recent form has a slightly negative coefficient, which is likely noise or mean reversion. The toss carries almost no weight, matching section 6.

Feature	Logistic coefficient (standardised)
home_diff	+0.219
elo_diff	+0.128
bat_first_edge	+0.098
career_diff	+0.054
toss_a	+0.036
form_diff	-0.076

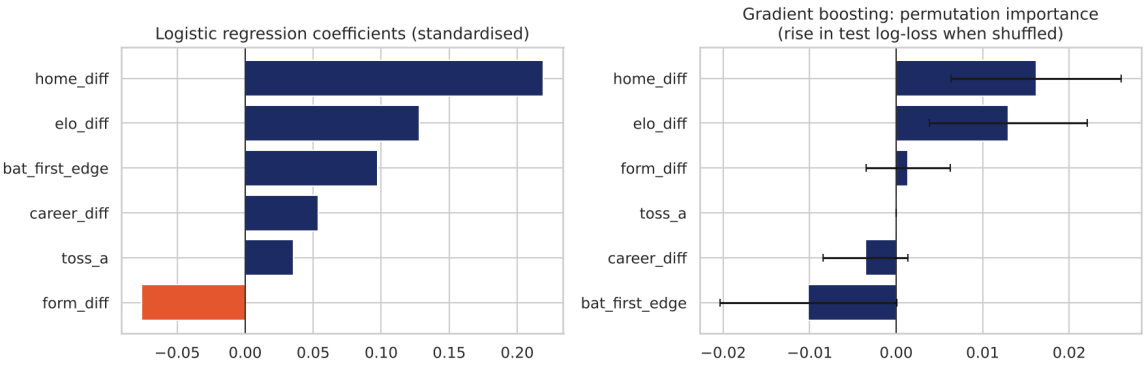


Figure 11. Logistic coefficients (left) and gradient boosting permutation importance (right) (chart 11).

7.12 Why the model barely beats a coin flip

- **T20 is high-variance.** One over can swing a match, and teams are closely matched. The Elo rating gap between teams has a standard deviation of only about 17 points; a 200-point gap would mean a 76% favourite.
- **Squads change every year** at the auction, so past results say less about the current team.
- **Missing information.** The model does not know who is playing, nor injuries, pitch conditions or weather. Squad strength is probably the biggest missing piece.
- **Small data.** About 1,000 training matches cannot support complex models; the evidence in this report is that they overfit.

8. The interactive dashboard

8.1 Purpose

Notebooks need to be run, or read on GitHub. The dashboard lets a reviewer explore the results in seconds without any code: choose a range of seasons, a team or a ground, and the charts and confidence intervals recalculate.

8.2 Architecture

- **One static file.** docs/index.html holds the page, styles, script and data. No server, no external libraries; it opens from a file and works on GitHub Pages.
- **Data compaction.** Each match becomes one array of ten numbers (season, team indices, winner, toss, venue, first-innings score, sixes, final flag). Per-season phase runs and overs, player runs and wickets by season, and the notebook 2 results are embedded as JSON. The page is about 130 KB.
- **Calculations in the browser.** Win rates, Wilson intervals, season averages, batting-first rates and top-ten lists are all computed live from the match records for whatever seasons are selected. The same Wilson formula is used as in the notebook.
- **Static results from the notebook.** The six hypothesis tests, model scores, walk-forward values and coefficients are read from results/results.json, which notebook 2 writes, so those figures cannot drift from the analysis. They do not change with the season filter, and the page says so.
- **Hand-drawn SVG charts.** Four chart types are built from small functions (line, vertical bar, horizontal bar, and dot with interval), reused across 15 charts. Charts redraw when the window resizes so text stays readable on phones.

8.3 What is on the page

Section	Contents
Filter bar	From and To season, four presets (all seasons, 2008 to 2022, Impact Player era, last five), season count
Summary tiles	Matches, average first-innings score, chasing win rate with CI, toss win rate with CI
Scoring	Average first-innings score, sixes per match, run rate by phase, with the 2023 Impact Player rule marked
Toss and chasing	Forest plot of toss and chasing rates with CIs and plain verdicts; chasing win rate by season; share of toss winners choosing to field
Teams	Franchise win rate ranking with titles; team picker with home-versus-away comparison and CI; win rate by season; batting-first win rate by ground with CI
Players	Top ten run scorers and wicket takers for the selected seasons
Which findings are real?	The six tests with Holm-adjusted p-values and plain-language notes
Can the winner be predicted?	Accuracy with intervals, walk-forward log-loss by season, logistic coefficients

8.4 Design decisions

- **Charts follow a few fixed rules:** thin marks, quiet gridlines, one y-axis, a legend whenever there are two or more series, and values at the end of lines. Hovering any mark shows exact numbers.
- **Colours:** series use blue, orange and green from a categorical order chosen to stay distinguishable for people with colour-vision deficiencies; CSK and Chepauk are highlighted in orange. The page has light and dark themes.
- **Verdicts never rely on colour alone.** "Real effect" and "No clear evidence" are pills with an icon and text.
- **Every chart has a "View as table" option** with the same numbers, so nothing depends on reading a graphic.
- **Small samples are shown, not hidden.** Ground and franchise charts require a minimum number of matches in the selected range and say what it is.
- **Typography:** Barlow Condensed for headings gives a scoreboard feel; Barlow for text; with system fallbacks if fonts do not load.

8.5 Rebuilding

```
# 1. re-run notebook 2 so results/results.json is refreshed
jupyter nbconvert --to notebook --execute --inplace ipl_inference_and_prediction.ipynb
# 2. rebuild the dashboard page
python scripts/build_dashboard.py
```

To publish: push to GitHub, open Settings, then Pages, and choose the main branch and the /docs folder. The dashboard is then served at a public address of the form https://YOUR-USERNAME.github.io/ipl-cricket-analysis/.

9. Verification performed

Check	Result
Dashboard figures against notebook outputs (matches, toss and chasing rates and intervals, franchise table, CSK record)	All match: 54.1% chasing, 50.8% toss, CSK 58.2% with 5 titles, 144 test matches
Franchise ranking and toss-field share against the original charts	Identical values
Top-ten player lists against an exact recomputation for every possible season range (153 ranges)	0 mismatches after lowering the inclusion thresholds (an earlier, stricter threshold would have dropped a genuine top-ten bowler in two single-season views, so it was fixed)
Clean-copy test: delete all charts, results and the dashboard, re-run both notebooks' code and the build script	Everything regenerated; results.json and index.html were byte-identical to the shipped versions
README links and images	All resolve
Dashboard rendering in headless Chromium at desktop and phone widths, light and dark	No script errors, no horizontal scrolling, 15 charts drawn; filters, team picker and hover tooltips exercised

What was not verified

- The notebooks' code was run with a small script that executes cells in order, because Jupyter could not be installed in the build environment. Opening both notebooks once in Jupyter on a normal machine is recommended.
- The GitHub Pages deployment was not tested, because it needs access to a GitHub account.
- The web fonts could not be fetched in the test environment, so the final typeface was not seen; the fallback fonts were.

10. Reproducibility

- **Fixed seed.** SEED = 42 is used for all random draws: permutation test, bootstrap, orientation randomisation and the gradient-boosting model.
- **Order matters.** One NumPy random generator is created in the setup cell and shared; the permutation test draws from it before the orientation randomisation does. Run all cells from top to bottom, in order, to reproduce the numbers exactly; running cells out of order can change the random orientation and therefore small details of the model results.
- **Time-ordered processing.** Matches are sorted by date and match id before ratings are updated, so results do not depend on the row order of the file.
- **Pinned minimums.** requirements.txt sets minimum versions for every package; exact versions used are listed in section 4.
- **Single source of truth.** The dashboard reads results.json rather than retyped numbers.

11. Limitations and threats to validity

- **Small groups.** Chepauk has 85 matches with a result, CSK at home 71, and the prediction test set 144. Intervals are wide, and small real effects cannot be confirmed or excluded.
- **Description is not causation.** The data shows what happened, not why. It cannot say why batting first works at Chepauk, or whether the Impact Player rule caused the 2023 scoring jump.
- **Home advantage confounded with team quality.** CSK is a strong team and also plays at Chepauk; the data cannot separate the two.
- **Franchise merging is a judgement.** Renamed teams are merged; Deccan Chargers and Sunrisers Hyderabad are not.
- **Elo settings tuned on 2016 to 2022.** The walk-forward results for those seasons are therefore slightly optimistic. The 2023 and 2024 hold-out is not affected.
- **No squad information.** The model does not know who is playing, which is probably the largest source of missed signal.
- **Multiple comparisons beyond the six tests.** Many descriptive comparisons were made in notebook 1; only the six tests were formally corrected.
- **Excluded data.** Super overs, five no-result matches and (for the scoring test) rain-affected innings are excluded.
- **Data licence.** The code is MIT licensed; the data belongs to its original sources and Cricsheet's terms should be checked before reuse.

Future work

- Add squad strength (players selected, auction value, recent individual form) as features; this is the biggest missing piece.
- Model first-innings score with a regression and report mean absolute error.
- Fit a hierarchical model of home advantage by team to separate ground effects from team quality.
- Check whether the 2023 jump holds up when adjusting for pitch and venue mix.
- Add automated tests for the data-cleaning and Elo functions.

12. Why this, why that

Why Wilson intervals instead of the simple formula?

The simple formula (p plus or minus 1.96 standard errors) can give limits below 0 or above 1 and is unreliable for small groups such as 71 home matches. Wilson intervals stay in range and have better coverage.

Why bootstrap, permutation and Fisher rather than one test for everything?

Each fits the question. Fisher is exact for small 2 by 2 counts. The permutation test answers "could random labels produce this gap?" without a formula. The bootstrap gives an interval for any difference. Mann-Whitney suits scores that are not perfectly normal.

Why correct for multiple testing, and why Holm?

Six tests mean a raw 5% threshold would flag a false positive about 26% of the time (1 minus 0.95 to the sixth power) if nothing were real. Holm controls that overall risk and is never less powerful than plain Bonferroni.

Why report confidence intervals as well as p-values?

A p-value says only whether an effect is distinguishable from zero. The interval shows how large it could be, which is what matters for a decision, and it makes clear when a sample is too small.

Why a time-based split?

A random split would leak information from the same season into training and overstate accuracy. Predicting the future means training on the past.

Why walk-forward evaluation as well?

A single 144-match test set is one draw and can flatter or punish a model by luck. Nine test seasons give 576 matches and show whether an edge is stable.

Why Elo?

It is a simple, interpretable strength rating that updates one match at a time and cannot leak future results. It is a strong baseline for head-to-head contests.

Why K = 5 and a 0.75 season carry-over?

They were chosen by log-loss on 2016 to 2022 matches over a grid of twelve settings, never on the test years. Slow updating won, which itself signals how little one match reveals about team strength.

Why logistic regression and gradient boosting, not deep learning or a random forest?

With about 1,000 noisy examples and six features, a regularised linear model is the right starting point and its coefficients can be read. Boosting tests whether non-linear effects help; it did not, and deep learning would only overfit.

Why log-loss and Brier score, not only accuracy?

Accuracy ignores confidence and, with 144 matches, is very noisy. Log-loss and Brier reward well-calibrated probabilities, which is what a forecasting model is for.

Why randomise which team is team A?

If the file tends to list the eventual batting-first team first, a model could learn that for free. Random orientation with a fixed seed removes the leak and makes the target balanced.

Why permutation importance rather than SHAP?

It is model-agnostic, built into scikit-learn and easy to explain: shuffle a feature and see how much worse the model gets. For six features it ranks them as well as SHAP would.

Is a model that barely beats a coin flip a failed project?

No. It is an honest finding supported by baselines, intervals and a leak-free evaluation. It matches what is known about T20: high variance and closely matched teams. A claim of 70% accuracy from pre-match data would be the red flag.

Why merge renamed franchises but keep Deccan Chargers and Sunrisers apart?

Delhi Daredevils and Delhi Capitals are the same franchise under a new name. Deccan Chargers and Sunrisers Hyderabad are different franchises that played in the same city, so merging them would mix two teams' results.

Why exclude super overs, no-result matches and rain-affected innings?

Super overs follow different rules and distort run rates, no-result matches have no winner, and shortened innings are not comparable with full ones in the scoring test.

Why plain HTML and JavaScript for the dashboard instead of Streamlit or Plotly?

A static file needs no server, never breaks when a dependency changes, and runs free on GitHub Pages. Hand-written SVG gave full control over intervals, tooltips and tables.

Why 10,000 bootstrap resamples, 20,000 permutations and 5,000 for the model intervals?

These are large enough that Monte Carlo noise in the intervals and p-values is small compared with the sampling uncertainty, while still running in about a minute.

Why 95%?

It is the convention and makes results comparable with other work. The full intervals are shown, so a reader can judge for themselves.

13. Glossary

Term	Meaning
Accuracy	Share of predictions that were correct.
Bootstrap	Resampling the data with replacement many times to see how much a statistic varies.
Brier score	Average squared difference between predicted probability and the 0 or 1 outcome.
Calibration	Whether events predicted at x% really happen about x% of the time.
Chasing	Batting second, with a target to beat.
Cohen's d	A standardised effect size: the difference in means in units of standard deviation.
Confidence interval	A range of values consistent with the data; wide when samples are small.
Death overs	Overs 16 to 20.
Elo rating	A strength rating that rises after wins and falls after losses, by more when the result was surprising.
Fisher's exact test	An exact test for association in a 2 by 2 table of counts.
Gradient boosting	An ensemble of small decision trees, each correcting the errors of the previous ones.
Holm-Bonferroni	A step-down correction that controls the chance of any false positive across several tests.
Impact Player rule	A rule introduced in the 2023 IPL season that lets teams substitute a player during a match.
Leakage	Using information in training or features that would not be available when a real prediction is made.
Log-loss	A score for probability forecasts that heavily penalises confident wrong answers.
Logistic regression	A model that predicts the probability of a yes/no outcome from a weighted sum of features.
Mann-Whitney U	A rank-based test of whether values in one group tend to be larger than in another.
Permutation test	A test that shuffles labels to see how often chance alone produces an effect as large as the observed one.
Permutation importance	The drop in model performance when one feature is shuffled.
Powerplay	Overs 1 to 6, with fielding restrictions.
Walk-forward evaluation	Repeatedly training on the past and testing on the next season.
Wilson interval	A confidence interval for a proportion that behaves well for small samples.

14. Talking points for a resume or interview

Resume bullets

- Analysed 1,095 IPL matches and about 260,000 deliveries (2008 to 2024) in Python (pandas, NumPy), cleaning inconsistent team and venue names and quantifying a +23-run jump in first-innings scoring in 2023 and 2024 (95% CI +17 to +29, Cohen's d = 0.75).
- Tested six claims about the toss, chasing and home advantage with exact, bootstrap and permutation tests and a Holm correction; found chasing has a small real edge (54.1%, CI 51.2% to 57.1%) while the toss shows no detectable effect (50.8%, CI 47.9% to 53.8%).
- Built and evaluated an Elo-based win predictor with leak-free features, a 2008 to 2022 / 2023 to 2024 time split and nine-season walk-forward validation; showed with bootstrap intervals that pre-match information barely beats a coin flip (best walk-forward accuracy 54.7%).
- Shipped a dependency-free interactive dashboard (HTML, CSS, JavaScript, SVG) that recomputes confidence intervals for any season range, deployable on GitHub Pages and rebuilt from the notebooks by a single script.

Three points to be ready to defend

- **The negative result.** Explain that high variance, closely matched teams and yearly squad changes make match prediction hard, and that baselines and intervals are what make the claim trustworthy.
- **Leakage control.** Be able to list the four protections: pre-match-only features, updates after each match, randomised team order, and tuning only on training years.
- **Honest uncertainty.** Be ready to say which findings are fragile (the two Chepauk results) and why, and that the data cannot show why anything happened.

Appendix A. Code for the key statistical helpers

Wilson interval

```
def wilson_ci(k, n, conf=0.95):
    z = stats.norm.ppf(0.5 + conf / 2)
    p = k / n
    centre = (p + z**2 / (2 * n)) / (1 + z**2 / n)
    half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / (1 + z**2 / n)
    return centre - half, centre + half
```

Percentile bootstrap for a difference

```
def bootstrap_diff(a, b, stat=np.mean, n_boot=10_000, seed=SEED):
    r = np.random.default_rng(seed)
    a, b = np.asarray(a), np.asarray(b)
    d = np.array([stat(r.choice(a, len(a))) - stat(r.choice(b, len(b))) for _ in range(n_boot)])
    return np.percentile(d, [2.5, 97.5])
```

Holm-Bonferroni correction

```
def holm(pvals):
    p = np.asarray(pvals)
    order = np.argsort(p)
    adj = np.empty_like(p)
    running = 0.0
    for rank, idx in enumerate(order):
        running = max(running, (len(p) - rank) * p[idx])
        adj[idx] = min(1.0, running)
    return adj
```

Permutation test for the Chepauk gap

```
obs = bf_chep.mean() - bf_other.mean()
pooled = np.concatenate([bf_chep, bf_other])
perm = np.empty(20_000)
for i in range(len(perm)):
    s = rng.permutation(pooled)
    perm[i] = s[:len(bf_chep)].mean() - s[len(bf_chep):].mean()
p_perm = (np.sum(np.abs(perm) >= abs(obs)) + 1) / (len(perm) + 1)
```

Evaluation of one walk-forward fold

```
for season in range(2016, 2025):
    tr, te = data[data.season < season], data[data.season == season]
    p = predictions(tr, te)           # refits logistic regression and gradient boosting on tr only
    # record accuracy and log-loss of every model on te
```